

Code Cultivation

オブジェクト指向ガーデンシステム

概要：Pythonの高度な概念を学びながら、包括的なデジタルガーデンエコシステムを構築します。データ駆動型のアプローチでコミュニティガーデンを効率的に管理するツールを作成します。

バージョン：3.1

目次

- I まえがき
- II AI利用ガイドライン
- III はじめに
- IV 全般的な指示
- V 演習0：最初の種を植えよう
- VI 演習1：ガーデンデータオーガナイザー
- VII 演習2：植物成長シミュレーター
- VIII 演習3：植物ファクトリー
- IX 演習4：ガーデンセキュリティシステム
- X 演習5：特化した植物タイプ
- XI 演習6：ガーデン分析
- XII 提出方法

第I章 まえがき

デジタル時代において、ガーデンにもスマートなシステムが必要です。

ある賢い庭師はかつてこう言いました：「花を植えておいて、根の様子を見るために毎日引っっこ抜くことはできない。」コードも同じです。プロセスを信頼し、プログラムが自然に、層を重ねながら育っていくことを許すとき、最も美しい成長が生まれることがあります。

すべての繁栄するコミュニティガーデンの背後には、関係性のネットワークがあります。植物は孤立して育つのではなく、各種が独自の貢献をしながら共通のニーズを分かち合うコミュニティを形成します。害虫から他の植物を守るものもあれば、土壌に窒素を固定するもの、繊細な苗に日陰を提供するものもあります。この相互支援の網は、よく設計されたソフトウェアシステムの動作様式を映し出しています。――固有の役割を持つ個別のコンポーネントが、調和して協力し合うのです。

熟練した庭師が健康な土壌が健康な植物を育てることを知っているように、経験豊かなプログラマーはよく構造化されたコードが堅牢なアプリケーションへと成長することを理解しています。コードを整理することはガーデンを設計することに似ています。――保護が必要な要素もあれば、成長する余地が必要な要素もあり、すべてが適切な場所と目的を持ったときに最良のシステムが生まれます。

このプロジェクトでは、コミュニティガーデンの発展に実際に役立てるツールを構築しながら、プログラミングスキルを磨いていきます。あなたが書くコードの一行一行は、可能性のデジタルな土壌に植えられた種です。

第II章 AI利用ガイドライン

● 背景

学習の旅の中で、AIはさまざまなタスクをサポートできます。AIツールのさまざまな機能と、それがどのように作業を支援できるかを時間をかけて探求してください。ただし、常に慎重にアプローチし、結果を批判的に評価してください。コード、ドキュメント、アイデア、技術的な説明のいずれであれ、質問が適切に形成されているか、生成されたコンテンツが正確かどうかを完全に確信することはできません。ピア（仲間）は、間違いや盲点を避けるための貴重なリソースです。

● 主なメッセージ

- ☒ AIを使って繰り返しの作業や単調な作業を減らしましょう。
- ☒ コーディングと非コーディングの両方のプロンプトスキルを磨きましょう——これは将来のキャリアに役立ちます。
- ☒ AIシステムの仕組みを学び、一般的ナリスク、バイアス、倫理的問題をより適切に予測・回避できるようにしましょう。
- ☒ ピアと協力して、技術スキルと社会スキルの両方を磨き続けましょう。
- ☒ 完全に理解し、責任を持てるAI生成コンテンツのみを使用しましょう。

● 学習者のルール

- ・ AIツールを探求し、その仕組みを理解する時間を取り、倫理的に使用し、潜在的なバイアスを減らせるようにしましょう。
- ・ プロンプトを入力する前に問題をよく考えましょう——これにより、正確な語彙を使った、より明確で詳細かつ関連性の高いプロンプトを書けるようになります。
- ・ AIが生成したものはすべて系統的にチェック・レビュー・疑問・テストする習慣をつけましょう。
- ・ 必ずピアレビューを求めましょう——自分自身の検証だけに頼らないようにしましょう。

● フェーズの目標

- ・ 汎用的かつドメイン固有のプロンプトスキルを身につける。
- ・ AIツールの効果的な活用で生産性を向上させる。
- ・ 計算論的思考、問題解決能力、適応力、協調性を引き続き強化する。

● コメントと例

- ・ 試験、評価など、真の理解を示さなければならない場面に定期的に遭遇します。準備を怠らず、技術スキルと対人スキルの両方を磨き続けましょう。
- ・ 自分の考えを説明したり、ピアとディスカッションしたりすることで、理解の不足している部分が明らかになることが多いです。ピア学習を優先しましょう。
- ・ AIツールはあなたの具体的な文脈を把握していないため、汎用的な回答を生成しがちです。同じ環境を共有するピアは、より関連性が高く正確な知見を提供できます。

- AIが最も確からしい答えを生成する傾向がある一方で、ピアは代替の視点や貴重なニュアンスを提供できます。品質チェックのよりどころとして活用しましょう。

✓ 良い例： AIに「ソート関数のテスト方法は？」と質問します。いくつかのアイデアをもらいます。試してみてもピアと結果をレビューします。一緒にアプローチを洗練させます。

☒ 悪い例： AIに関数全体を書いてもらい、そのままプロジェクトにコピー&ペーストします。ピア評価のとき、何をしているのか、なぜそうするのか説明できません。信頼を失い――プロジェクトに失敗します。

✓ 良い例： AIを使ってパーサーを設計します。その後、ピアとロジックを確認します。バグを2つ発見し、一緒に書き直します――より良く、よりクリーンで、完全に理解した状態で。

☒ 悪い例： Copilotにプロジェクトの重要な部分のコードを生成させます。コンパイルは通りましたが、パイプ処理の仕組みを説明できません。評価で説明に失敗し、プロジェクトに落ちます。

第III章 はじめに

CodeCultivationへようこそ！

最初のアクティビティで学んだPythonの基礎を土台に、包括的なガーデンデータ管理システムを構築することで、より複雑なプログラミングの課題に挑戦します。このプロジェクトでは、Pythonが現実世界のシステムをモデル化する強力なツールとなる高度な概念を紹介します。

取り組む内容：

- Pythonプログラムがどのように構造化され、実行されるかを理解する
- オブジェクト指向のアプローチを使ってデータを整理する
- 再利用可能なコードコンポーネントを作成する
- 適応・拡張できるシステムを構築する
- 共同作業環境でのデータの整合性を守る
- スケーラブルなソフトウェアアーキテクチャを設計する

各演習は前の演習を基礎として積み上げ、最終的に完全なデジタルガーデンエコシステムを完成させます。

【重要】このモジュールはPythonプログラムの基本構造から始まり、オブジェクト指向プログラミングへと進んでいきます。各演習には、要求された定義と必要なコードを含めてください。テスト用に `if __name__ == "__main__":` ブロックを各ファイルの末尾に含めることができます。

第IV章 全般的な指示

- コードはPython 3.10以上で記述すること
- コードはflake8リントー基準に準拠すること
- 各演習は独自のファイルとディレクトリに配置すること
- 適切な命名規則を使用すること：クラスはPascalCase、関数と変数はsnake_case
- すべての関数とメソッドには型ヒントを含めること：mypyでコードを検証すること
- 明示的に指示されない限り、入力値の検証は不要
- プログラミングの概念を明確に示すことに集中すること
- プログラムは常にエラーなく実行されること

【デジタルガーデンエコシステムについて】このプロジェクトはPythonプログラミングの概念に焦点を当て、基本的なプログラム構造からオブジェクト指向プログラミングへと進んでいきます。各演習では新しい機能を導入しながら、一貫性のあるガーデン管理システムを構築します。後の演習では前の演習の概念を再利用します。

【オブジェクト指向プログラミングについて】演習1以降、このモジュールのすべての演習でクラスの使用が必要です。「class」や「def」などのPythonキーワードは基本的な言語キーワードであり、各演習の「使用可能な関数」リストに記載する必要はありません。

表示を見やすくするために、str（文字列）オブジェクトのcapitalize()メソッドを使用できます。これは「使用可能な関数」には含まれませんが、必要に応じて使用して構いません。

第V章 演習0：最初の種を植えよう

演習名	Exercise0 / ft_garden_intro
ディレクトリ	ex0/
提出ファイル	ft_garden_intro.py
使用可能な関数	print()

複雑なガーデン管理システムを構築する前に、Pythonプログラムがどのように機能するかを理解する必要があります。すべてのPythonプログラムには出発点——実行が始まる場所——が必要です。ガーデンに最初の種を植えるようなものです！

ガーデンの植物に関する情報を表示する、最初のPythonプログラムを作成してください。

要件：

- `python3 ft_garden_intro.py` で直接実行されるプログラムを作成すること
- 特別な `if __name__ == "__main__":` パターンを使用すること
- 植物情報をシンプルな変数に保存すること：name（名前）、height（高さ）、age（樹齢）
- `plant()` を使用して植物情報を表示すること
- 以下に示す出力表示を模倣すること（評価では厳密な出力チェックは行いません）

実行例：

```
$> python3 ft_garden_intro.py === Welcome to My Garden === Plant: Rose Height: 25cm Age: 30 days ===  
End of Program ===
```

クラスなどのオブジェクト指向の機能などのより複雑な概念に移る前に、プログラムがどのように開始・実行されるかを理解することが基本です。

`if __name__ == "__main__":` の行を使用することが重要な理由を説明できる必要があります。これはインポートの概念につながります。今の時点でこの良い習慣を導入しておくことが重要です。

【ヒント】Pythonファイルが `#!` で始まる特別な行（「シェバング」行）で始まることに気づいたことはありますか？この行が何をするのか、スクリプトを直接実行可能にするためにどのように役立つかを調べてみてください。評価では、このシェバングを使用するようにスクリプトをライブで更新することを求められます。

第VI章 演習1：ガーデンデータオーガナイザー

演習名	Exercise1 / ft_garden_data
ディレクトリ	ex1/
提出ファイル	ft_garden_data.py
使用可能な関数	print()

コミュニティガーデンでは、複数の植物とその情報を追跡する必要があります。複数の植物のデータを効率的に保存・表示する必要があります。

各植物が持つ情報：

- 名前 (name)
- 高さ (センチメートル)
- 樹齢 (日数)

少なくとも3種類の異なる植物のデータを管理し、整理された形式で表示するプログラムを作成してください。

【必須】各植物を個別に処理するのではなく、植物の雛形となるPlantクラスを作成する必要があります。すべての植物は同じ属性（名前・高さ・樹齢）で記述されます。各植物についてクラスをインスタンス化し、属性に具体的な値を設定してください。

植物情報を表示するshow()メソッドをクラス内に作成してください。

実行例：

```
$> python3 ft_garden_data.py == Garden Plant Registry == Rose: 25cm, 30 days old Sunflower: 80cm, 45 days old Cactus: 15cm, 120 days old
```

現在、植物の情報はどのように保存していますか？複数のやり方があります。植物が増えたとき、どのような課題が生じる可能性があるでしょうか？次の演習では、アプローチを発展させる必要があるかもしれません。

第VII章 演習2：植物成長シミュレーター

演習名	Exercise2 / ft_plant_growth
ディレクトリ	ex2/
提出ファイル	ft_plant_growth.py
使用可能な関数	print(), range(), round()

ガーデンマネージャーは時間経過に伴う植物の成長をシミュレーションしたいと考えています。植物の変化を追跡し、状態を変更するための操作を提供する必要があります。

要件：

- 演習1のPlantクラスを再利用すること
- 植物がgrow()とage()を自分自身で行えるようにすること（これらはクラスのメソッドになります）
- 植物の1週間の成長をシミュレーションし、クラス内のデータにアクセスして最終的な高さを取得し、1週間の合計増加量を表示すること
- grow()とage()を使用したとき、植物がどのように変化するかを考慮すること。植物によって異なる成長をしても構いません

実行例：

```
$> python3 ft_plant_growth.py === Garden Plant Growth === Rose: 25.0cm, 30 days old === Day 1 ===  
Rose: 25.8cm, 31 days old === Day 2 === Rose: 26.6cm, 32 days old ... === Day 7 === Rose: 30.6cm, 37  
days old Growth this week: 5.6cm
```

これを処理するには複数のアプローチがあります。Plantクラスに属性を追加したり、クラス内のメソッドにパラメータを追加したりするなど、自分のやり方で実装して構いません。

第VIII章 演習3：植物ファクトリー

演習名	Exercise3 / ft_plant_factory
ディレクトリ	ex3/
提出ファイル	ft_plant_factory.py
使用可能な関数	print(), range(), round()

ガーデンセンターでは、Plantクラスを使って異なる初期値を持つ多くの植物を素早く作成する必要があります。植物の生成プロセスを効率化する必要があります：新しい植物のクラスのインスタンス化と初期化を同時に行います。

要件：

- 植物は初期情報（名前、初期の高さ、初期の樹齢）を直接指定して作成できること
- 作成直後に使用できる状態であること（例：grow()を呼び出せる）
- 異なる特性を持つ少なくとも5種類の植物を作成すること
- 作成したすべての植物を整理された形式で表示すること

実行例：

```
$> python3 ft_plant_factory.py == Plant Factory Output == Created: Rose: 25.0cm, 30 days old  
Created: Oak: 200.0cm, 365 days old Created: Cactus: 5.0cm, 90 days old Created: Sunflower: 80.0cm,  
45 days old Created: Fern: 15.0cm, 120 days old
```

show()メソッドをそのまま使用して上記の出力を表示できるはずです。

第IX章 演習4：ガーデンセキュリティシステム

演習名	Exercise4 / ft_garden_security
ディレクトリ	ex4/
提出ファイル	ft_garden_security.py
使用可能な関数	print(), range(), round()

ガーデンマネージャーはデータの整合性を心配しています。一部のボランティアが誤って無効な値（負の高さ、ありえない樹齢）を設定して植物データを破損させてしまいました。機密データを保護してカプセル化する安全なシステムを作成する必要があります。

要件：

- データの破損を防ぐためにPlantクラスを改善すること
- 植物データを変更するための制御された方法を提供すること：set_height()、set_age()
- 植物データに安全にアクセスするための方法を提供すること：get_height()、get_age()
- バリデーションにより植物の高さが負にならないようにすること
- バリデーションにより植物の樹齢が負にならないようにすること
- 無効な値が提供された場合、クラスからエラーメッセージを出力し、データを変更しないか、デフォルト値で植物を作成すること
- カプセル化を使用すること：「protected」規約（マングリングではない）を使用してクラス属性への直接アクセスを防ぐこと

実行例：

```
$> python3 ft_garden_security.py == Garden Security System == Plant created: Rose: 15.0cm, 10 days old
Height updated: 25cm Age updated: 30 days Rose: Error, height can't be negative Height update rejected
Rose: Error, age can't be negative Age update rejected Current state: Rose: 25.0cm, 30 days old
```

第X章 演習5：特化した植物タイプ

演習名	Exercise5 / ft_plant_types
ディレクトリ	ex5/
提出ファイル	ft_plant_types.py
使用可能な関数	super(), print(), range(), round()

ガーデンでは異なる種類の植物を扱う必要があります：花、木、野菜。各タイプは独自の特性を持ちながら、親カテゴリから共通の植物の機能を継承します。

要件：

- 前の演習のPlantクラス（共通機能：名前・高さ・樹齢）を出発点とすること
- 特化したタイプを作成すること：Flower（花）、Tree（木）、Vegetable（野菜）
- 各特化タイプは基本的な植物の機能を継承すること
- Flowerに必要なもの：color属性とbloom()メソッド
- Treeに必要なもの：trunk_diameter属性とproduce_shade()メソッド
- Vegetableに必要なもの：harvest_season属性とnutritional_value属性
- 特化した植物を作成する際、super()を使って新しいクラスの内側から親メソッドを呼び出すこと（__init__()を含む任意のメソッドに適用可能）
- 特化クラスでshow()を呼び出すと、標準的なPlantの出力と特化植物の追加特性が表示されること。メソッドのオーバーライドでは親の既存コードを再利用できること
- 各植物タイプのインスタンスを少なくとも1つ作成すること；花を開花させること；野菜のnutritional_valueは0から始め、age()とgrow()を呼ぶたびに増加させること
- 異なる特化タイプで共通の植物コードを重複させないこと
- 3つの新しいクラスで新しい属性のバリデーションは不要

実行例（抜粋）：

```
$> python3 ft_plant_types.py === Garden Plant Types ===  
=== Flower Rose: 15.0cm, 10 days old Color: red  
Rose has not bloomed yet [asking the rose to bloom] Rose: 15.0cm, 10 days old Color: red  
Rose is blooming beautifully! === Tree Oak: 200.0cm, 365 days old Trunk diameter: 5.0cm  
Tree Oak now produces a shade of 200.0cm long and 5.0cm wide. === Vegetable Tomato: 5.0cm, 10 days old  
Harvest season: April Nutritional value: 0 Tomato: 47.0cm, 30 days old Harvest season: April  
Nutritional value: 20
```

第XI章 演習6：ガーデン分析

演習名	Exercise6 / ft_garden_analytics
ディレクトリ	ex6/
提出ファイル	ft_garden_analytics.py
使用可能な関数	super(), print(), range(), round(), staticmethod(), classmethod()

前の演習のクラスを拡張して分析機能を表示します。複雑なデータ関係を扱い、ネストされたコンポーネントと継承チェーンを使用します。

要件：

- Plantクラスに静的メソッドを作成すること：パラメータとして与えられた特定の樹齢が1年を超えるかチェックする
- クラスメソッドを作成すること：すべての情報がまだ揃っていない場合に「匿名」の植物を直接作成できる
- SeedクラスをFlowerから継承して作成すること：花が開花した後の種の数を持し、show()メソッドを改善すること
- 各Plantはネストされたクラスとして実装された内部システムを持つこと：grow()の呼び出し回数、age()の呼び出し回数、show()の呼び出し回数という統計データを保持する。カプセル化と表示関数が必要
- Treeには追加の統計データが必要：produce_shade()の呼び出し回数
- 最後に、任意の種類の植物の統計を表示するための、クラスに属さない独自の関数を作成すること

静的メソッドとクラスメソッドにはデコレータ構文も使用可能です。

実行例（抜粋）：

```
$> python3 ft_garden_analytics.py == Garden statistics == == Check year-old Is 30 days more than a year? -> False Is 400 days more than a year? -> True == Flower Rose: 15.0cm, 10 days old Color: red Rose has not bloomed yet [statistics for Rose] Stats: 0 grow, 0 age, 1 show ... == Anonymous Unknown plant: 0.0cm, 0 days old Stats: 0 grow, 0 age, 1 show
```

この演習はプロジェクト全体で学んだすべてのプログラミングパターンを統合します。異なるコンポーネントがどのように相互作用し、複雑なシステム内でどのように整理されるかの理解について評価されます。

第XII章 提出方法

通常通り、Gitリポジトリに課題を提出してください。防衛（評価）中に評価されるのはリポジトリ内の作業のみです。ファイル名が正しいことを必ず確認してください。また、flake8とmypyを使ってPython標準と型ヒントも再確認してください。

評価中には、プログラミングの概念を説明したり、継承の関係を示したり、新しい機能でコードを拡張したりすることを求められる場合があります。実装の背景にある原則を理解しておくようにしてください。

【重要】提出するのは本プロジェクトの課題で要求されているファイルのみです。プログラミングの原則を明確に示す、クリーンでドキュメント化されたコードに集中してください。